How Our Model Actually Works

A plain-English walkthrough of every step the strategy takes, every month.
Project ml-factor-investing-2026 | 2026-05-22

The one-paragraph version

Every month-end we have ~500 stocks (the S&P 500 of that day). For each stock we compute 8 numbers describing it — its momentum, its size, its volatility, etc. We feed those numbers into a machine-learning model (XGBoost) that has been trained on 10 years of past examples. The model spits back a score for each stock — a guess about how it'll do next month. We buy the top-scoring 20% of stocks and sell-short the bottom 20%. Equal money in each. We hold those positions for one month, then redo the whole process. The model gets re-trained each month using the most recent 10 years of data.

That's it. Six paragraphs below explain each step.

Step 1. The investable universe

At the start of every month, we ask: which stocks are even eligible to buy or short today?

Rule: a stock must satisfy ALL of these on the rebalance date:

- **It was an S&P 500 member that day.** Not today — *that day*. So in 2008 we trade Lehman Brothers; in 2020 we trade ExxonMobil even though oil has fallen out of favour by 2024. This is called "point-in-time" membership and it prevents survivorship bias.
- **It has a price that day** (i.e. it's still trading; not delisted yet).
- **It has values for every one of the 8 features.** No missing data — if even one feature is NaN, the stock drops out for that month.
- **The model produced a non-NaN score for it.**

> **Typical month:** ~500 stocks pass all four checks. Some months a few more (e.g., during the dot-com era, when index churn was high) and some months a few less. Over the 2005-2024 window the universe ranges from ~480 to ~510 names per month.

Step 2. What we know about each stock

For every eligible stock, we compute 8 features. Think of these as the model's input: an 8-dimensional snapshot of the stock at this moment.

#	Feature	What it captures	How it's computed
1	Momentum (mom)	Is the stock on a winning streak?	Cumulative return from 12 months ago to 1 month ago (the
2	Short-term reversal (rev)	Did the stock just over-react?	Last month's return
3	Monthly volatility (mvol)	How shaky is the price?	Std dev of monthly returns over last 6 months
4	Idiosyncratic vol (ivol)	How shaky after removing market moves?	Std dev of OLS residuals over last 24 months
5	Log market cap (log_mktcap)	How big is the company?	log(shares outstanding × price)
6	Book-to-market (bm)	Is the company "cheap" by accounting yardstick?	Equity / market cap (Sharadar quarterly)
7	Earnings yield (ep)	Are earnings high relative to price?	TTM net income / market cap (Sharadar trailing 12mo)
8	Dollar volume (dvol)	How liquid is the stock?	21-day trailing average of price × volume (yfinance daily)

Critical step: sector-relative ranking

Raw numbers aren't comparable. A utility stock's volatility looks low next to a tech stock's. So for every feature, we convert the raw value into a **percentile rank within the stock's sector**:

> Apple has momentum = +30% over the past year. Other Information-Technology stocks that month range from -20% to +45%. Apple's *rank* inside IT is 0.85 — top 15% of its sector. That 0.85 is what the model sees, not the raw 0.30. This way a +30% return in a sleepy sector and a +30% return in a hot sector get fair treatment.

Every one of the 8 features goes through this rank step. The model only ever sees numbers between 0 and 1.

Step 3. The model predicts next-month return

Now we have, for each eligible stock, 8 ranks in [0, 1]. We feed those 8 numbers into our trained XGBoost model and it returns a single number per stock — its **predicted return** for the next month.

What XGBoost actually does (in 30 seconds)

XGBoost is a collection of 150 small decision trees that were trained together. Each tree asks questions like "is momentum above 0.7? then add 0.005 to the prediction" → "is book-to-market below 0.3? subtract 0.003". You add up all 150 trees' contributions and that's the prediction.

It's not magic — it's a fancy weighted average of if-then-else rules that the training process built up automatically.

Concrete example (made up but realistic-looking):

Stock: NVDA on 2023-06-30.
Sector-relative ranks: mom 0.95 (high), rev 0.40, mvol 0.80, ivol 0.75, log_mktcap 0.98, bm 0.10 (low — "expensive"), ep 0.55, dvol 0.99.

Model output (the prediction): **+0.012** — "NVDA should beat the median S&P 500 stock by about 1.2% next month".

Stock: T (AT&T) on the same date.
Ranks: mom 0.20, rev 0.60, mvol 0.30, ivol 0.25, log_mktcap 0.85, bm 0.75, ep 0.65, dvol 0.70.

Model output: **-0.004** — "AT&T should under-perform the median by ~0.4% next month".

These numbers aren't precise return forecasts — they're more like a **score**. What matters is the **ranking**: which stocks scored high, which scored low.

Step 4. Decide who to buy, who to short, who to skip

We take all ~500 scores and sort them from highest to lowest.

Then we draw two cut lines:

- **Long basket** — the top 20% of scores. We buy those stocks. ~100 of them.
- **Short basket** — the bottom 20% of scores. We sell-short those stocks. ~100 of them.
- **The middle 60%** — we hold no position. ~300 stocks are skipped that month.

So out of ~500 eligible stocks each month, we typically have **~200 active positions**: half long, half short.

Why skip the middle?

The model's confidence is highest at the extremes. If a stock scored just barely above average, the model's prediction is basically noise. Buying it would add transaction cost without adding meaningful information. So we only act on the high-conviction picks — the most-extreme 40% of names.

Step 5. How much money goes into each position?

Equal weight within each basket. If we have 100 longs, each one gets 1/100 of the long capital. Same for shorts. No stock is special — Apple gets the same dollar exposure as a tiny mid-cap that happened to make the top 20%.

Dollar-neutral, gross exposure 2x.

	Long basket	Short basket	Total
Number of stocks	~100	~100	~200 positions
Weight per stock	+1.0%	-1.0%	
Sum of weights	+100%	-100%	0% net market exposure
Absolute exposure	100%	100%	200% gross (2x leverage)

If you started with $1,000:
Long $1,000 in 100 stocks = $10 per long position.
Short $1,000 in 100 stocks = -$10 per short position.
Cash position: still $1,000 (since shorting gives you cash).
You owe $1,000 of stock you've borrowed-and-sold, but you have $1,000 of stock you bought, plus your original $1,000 of cash — net assets unchanged. Profit/loss comes from the spread between the two baskets.

Why this design?

- **Dollar-neutral by construction** (note: not exactly market-neutral -- see DOLLAR_VS_BETA_NEUTRAL.pdf for the full discussion). If the whole market goes up 5%, both baskets are up roughly 5%. Our P/L = (long leg's return) - (short leg's return). We make money only if our top-20% outperforms our bottom-20%, regardless of which direction the market went -- though the long leg systematically loads on higher-beta names, so a small positive net beta (around +0.2 to +0.4) leaks in.
- **No bet on direction.** We're not trying to time the market. We're betting we can pick winners *relative to losers*.
- **Transaction-cost-aware.** Equal-weight is robust to noisy predictions — you don't get whacked by a single big position turning bad.

Step 6. Trade and pay transaction cost

From last month's portfolio to this month's portfolio, some stocks moved in (entered top/bottom 20%), some moved out, some are still there.

We compute **turnover**: how much of the portfolio actually changed. Then we charge a transaction cost of **10 basis points (0.10%) per dollar traded**. This covers the bid-ask spread plus market impact for liquid S&P 500 names.

Typical turnover, our strategy: ~1.8 per month (i.e., we replace ~90% of our positions month over month). Transaction cost on each rebalance ≈ 1.8 × 10 bps = 0.18% of NAV. Over a year that's ~2.16% of return given to costs.

What the cost charges in practice: if last month we were long AAPL at +1% weight and this month AAPL drops out of the long basket, we pay 10 bps on the 1% closeout — 0.0001 of NAV. Multiply by 200ish trades and you get ~0.2% of NAV in cost per month.

Step 7. How the model gets its rules in the first place

So far we've described what happens at a single rebalance date. But how does the XGBoost model *know* which feature combinations predict positive returns? Training.

The training procedure

- **Look back 10 years.** If we're rebalancing on 2020-01-31, the training data is every (stock, month) pair from 2010-01 to 2019-12. That's ~500 stocks × 120 months ≈ 60,000 training examples.
- **For each training example, we have the 8 features as of that month AND the realised next-month return.** The model learns: "when features looked like X, the stock returned Y on average".
- **XGBoost fits 150 decision trees to that data.** Each new tree learns to predict the prediction error of the previous trees, so the ensemble keeps improving.
- **The fitted model is used to predict at 2020-01-31**, and only that one date. Next month (2020-02-29) the whole process repeats: re-train on 2010-02 to 2020-01, predict at 2020-02-29.

This is called a **walk-forward backtest**. The model never sees data from the future — only the past. Each rebalance gets a fresh model trained on its own 10-year window.

> **Why 10 years (and not, say, 3 or 20)?**
> 10 years is roughly one full business cycle: an expansion plus a recession. Shorter training windows miss recession patterns; longer ones include data so old it's no longer representative. 120 months is the conventional choice in the Gu-Kelly-Xiu (2020) paper this project replicates.

Step 8. How we set the model's hyperparameters

XGBoost has lots of dials: how many trees, how deep, how fast to learn, how much to regularise, etc. Picking those by gut feel would be cheating (you'd inadvertently tune them to look good on the test data).

So we used a clean three-way split of the timeline:

Sample	Dates	What we used it for
Training	2005-01 → 2015-12	The model fits its parameters on this window
Validation	2016-01 → 2018-12	Optuna searched for the best XGBoost hyperparameters on this window (6
Testing	2019-01 → 2024-12	Sacred. Never touched until we're ready to compute the final report numbe

The hyperparameters Optuna picked:

Hyperparameter	Textbook default	Tuned value	Effect
n_estimators	300 trees	150 trees	Half the size
max_depth	6	4	Shallower trees
learning_rate	0.10	0.015	Much slower

min_child_weight	1	15	Larger leaves
reg_alpha (L1)	0	0.40	L1 penalty added
reg_lambda (L2)	1	2.85	L2 tightened

The whole pattern: **more regularised, slower learning, smaller forest**. This is what you'd expect for a signal-to-noise-low problem like stock-return prediction. Letting the model be aggressive would just overfit noise.

The bottom line — what the strategy actually delivered

Out-of-sample, on the 2019-2024 test window the model never trained on (or touched during hyperparameter tuning):

Metric	Value	Plain English
Net Sharpe	+0.53	Return per unit of risk. >0 means we beat the risk-free rate on a risk-adjusted bas
Annualised return	+4.86%	Net of 10 bps transaction cost. Compounded over 5 years = ~+27% cumulative.
Max drawdown	-14%	Worst peak-to-trough loss. For comparison the S&P 500 dropped ~35% in th
Information coefficient	+0.0067	How well the model ranks stocks. Tiny but positive across 5 years — most produ
Average turnover	1.82	We replace ~90% of positions each month. High turnover, but absorbed by the co

How this compares to just buying the index.
S&P 500 over the same 2019-2024 window: ~+13% per year annualised, max drawdown -34% (COVID + 2022). Our strategy: +4.86%/year, max drawdown -14%. We make less money but with less than half the maximum loss. A risk-adjusted comparison (Sharpe) is the fair one — our 0.53 beats the index's ~0.5 for the same window, while being market-neutral (uncorrelated with what the market did).

Common questions, answered briefly

Q. We use 8 features, but the spec says ML can use 94. Why so few?

Two reasons. (1) Data: 86 of GKX's 94 features need fundamentals we'd have to buy access to. (2) Simplicity: GKX's own analysis shows the top 10 most-important features are nearly all price-based (momentum, volatility, size) — adding 80 more features barely changes Sharpe. The 8 we chose cover the four categories the paper identifies as load-bearing: trend, liquidity, volatility, value.

Q. Why long-short and not just long the top 20%?

Two answers. (1) Market neutrality: shorting hedges out the "will the market crash this month?" risk. (2) The model's predictions are *relative* — "stock A will beat the median" is a different claim from "stock A will earn 5%". Going long-short captures the relative-ranking signal cleanly.

Q. What happens if a stock in our long basket goes bankrupt?

It realises the actual return — typically -99% or so — and we eat the loss. Bankruptcies in our universe (Lehman 2008, SVB 2023, etc.) are not back-filled, smoothed, or removed. This is the survivorship rule we enforce in Step 1.

Q. Is this strategy actually tradeable in practice?

In principle yes — S&P 500 stocks are very liquid and 10 bps is a realistic cost assumption. The 1.8x monthly turnover is high but not extreme. The bigger practical concern is that factor strategies decay over time as more investors discover them, so historical Sharpe is not a guarantee.

Q. The model thinks about each stock independently. Doesn't that miss interactions?

Yes — that's why XGBoost is preferred over Lasso. Decision trees naturally pick up interactions like "high momentum AND low volatility predicts higher returns than either alone". Lasso can't, because it adds features linearly. This is exactly what we observed in our numbers: XGBoost +0.53 Sharpe vs Lasso +0.04 Sharpe on the same features.

Q. What's NOT in the model?

- Earnings announcement timing (we don't know when the next earnings call is).
- Analyst forecasts (we'd need a separate paid feed).
- Macro indicators (those go into Person C's separate regime model that scales our position sizes; not an input to the stock picks themselves).
- Anything about the market direction or VIX level. The model is time-blind and sector-blind by construction.